Sorting Algorithms — Exam Cheat Sheet

Visual Edition · Source: Lecture slides on Binary Search, Bubble / Selection / Insertion / Merge / Quick / Counting / Radix Sort. Scope: algorithms, C++ implementations, complexity analysis, common pitfalls, and probable exam questions.

1. Quick Reference — All Algorithms at a Glance

Comparison sorts are bounded below by Ω(n log n). Non-comparison sorts (Counting, Radix) break this barrier by exploiting key structure (integer range, digit position).

Algorithm	Type	Best	Average	Worst	Space	Stable
Binary Search	Search (D&C;)	Θ(1)	Θ(lg n)	Θ(lg n)	O(1) iter / O(lg n) rec	n/a
Bubble Sort	Comparison	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection Sort	Comparison	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion Sort	Comparison	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge Sort	Comparison (D&C;)	O(n lg n)	O(n lg n)	O(n lg n)	O(n)	Yes
Quick Sort	Comparison (D&C;)	O(n lg n)	O(n lg n)	$O(n^2)$	O(lg n)	No
Counting Sort	Non-comparison	O(n+k)	O(n+k)	O(n+k)	O(n+k)	Yes
Radix Sort	Non-comparison	O(d·(n+b))	O(d·(n+b))	O(d·(n+b))	O(n+b)	Yes

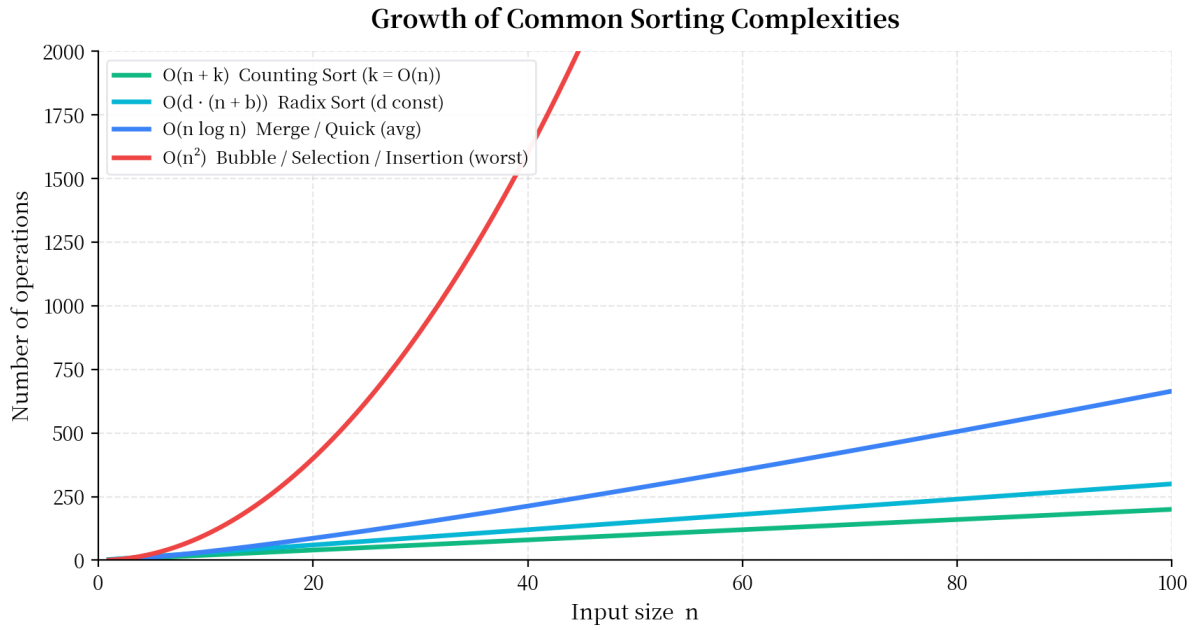


Figure 1.1 — Growth comparison of common sorting complexities. $O(n^2)$ sorts explode quickly; O(n log n) is the comparison-sort floor.

1.1 Decision Tree Lower Bound (why comparison sorts cannot beat n log n)

- **Theorem:** Any comparison sort requires Ω(n log n) comparisons in the worst case.
- **Proof sketch:** A comparison sort on n elements has a binary decision tree with n! leaves (one per permutation). If tree height is h, then $2^h \geq n!$, so $h \geq \lg(n!) = \Omega(n \log n)$.
- **Consequence:** Merge Sort and Heap Sort are asymptotically optimal comparison sorts. Quick Sort is optimal on average.
- **Escape route:** Use information other than comparisons (e.g., key value as array index in Counting Sort).

1.2 Divide & Conquer Template (used by Binary Search, Merge Sort, Quick Sort)

- **Divide:** split the problem into smaller sub-problems of the same type.
- **Conquer:** solve sub-problems recursively (base case: small enough to solve directly).
- **Combine:** merge sub-problem solutions into a solution for the original problem (may be empty — e.g., Quick Sort).
- **Generic recurrence:** T(n) = a·T(n/b) + f(n); for Merge Sort, a=2, b=2, f(n)=Θ(n) ⇒ T(n) = Θ(n log n).

2. Binary Search

2.1 Basic Idea

- Searches for a key in a sorted array by repeatedly halving the search interval.
- Textbook example of Divide & Conquer (combine step is empty).
- Requires the input to be sorted; otherwise undefined.
- Time Θ(lg n); space O(1) iterative or O(lg n) recursive (call stack).

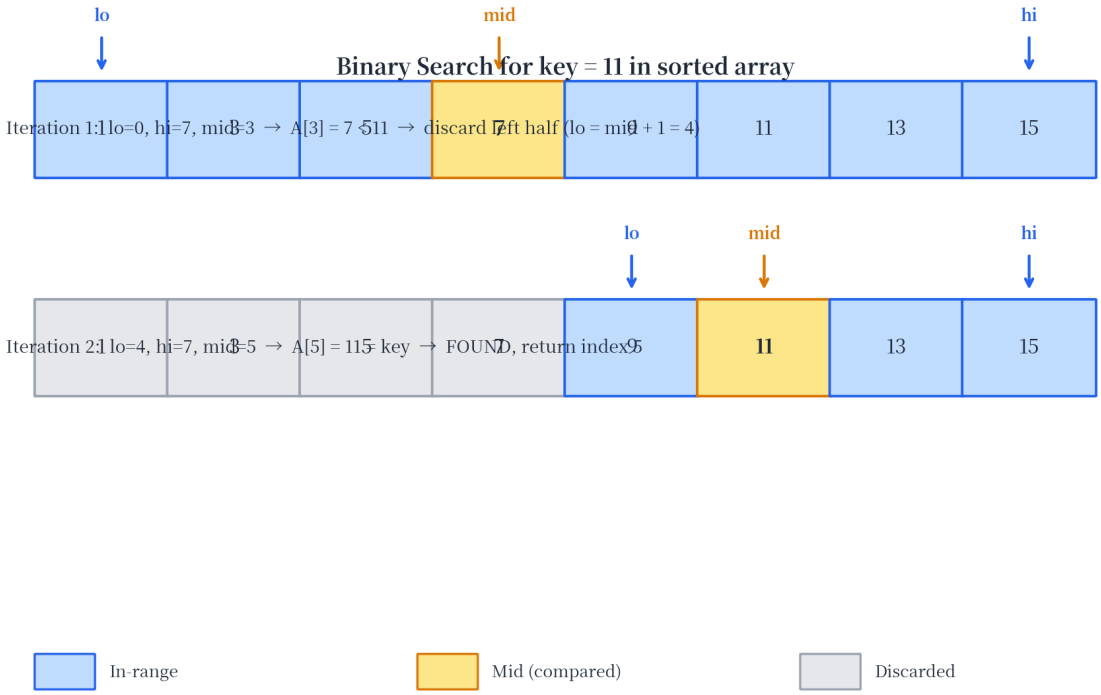


Figure 2.1 — Binary search for key=11 in a sorted array of 8 elements. Each iteration halves the search range; only 2 of lg(8)=3 iterations shown.

2.2 Algorithm (Recursive)

Objective: find index of x in sorted A[first..last]. **Input:** sorted A, indices first ≤ last, key x. **Output:** index of x or -1.

```
BinarySearch(A, first, last, x)
1. if last >= first then
2. mid = first + (last - first) / 2 // avoids overflow
3. if A[mid] == x return mid
4. else if A[mid] > x return BinarySearch(A, first, mid-1, x)
5. else return BinarySearch(A, mid+1, last, x)
6. return -1 // not found

// Initial call: BinarySearch(A, 0, n-1, key)
```

- **Observation:** only one sub-problem is explored per call ⟹ recurrence $T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$.
- **Why lg n:** each comparison discards half of the remaining elements; n elements ⟹ at most lg n halvings.

2.3 C++ Implementation

```
// Iterative (preferred: O(1) space)
int binarySearch(const vector& A, int x) {
    int lo = 0, hi = (int)A.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // overflow-safe
        if (A[mid] == x) return mid;
        else if (A[mid] > x) hi = mid - 1;
        else lo = mid + 1;
    }
    return -1; // not found
}

// Recursive variant
int bsRec(const vector& A, int x, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (A[mid] == x) return mid;
    if (A[mid] > x) return bsRec(A, x, lo, mid - 1);
    return bsRec(A, x, mid + 1, hi);
}
```

2.4 Common Mistakes

- Using `mid = (lo+hi)/2` — integer overflow for large arrays. Use `lo + (hi-lo)/2`.
- Wrong loop condition (`lo < hi` vs `lo ≤ hi`) — misses single-element range.
- Forgetting that the array must be sorted first.
- Off-by-one in `hi = mid-1` / `lo = mid+1` — using mid directly causes infinite loop.

3. Bubble Sort

3.1 Basic Idea

- Repeatedly compare adjacent pairs; swap if out of order. After pass i, the i-th largest element is in its final position.
- Largest values "bubble" to the end; smallest "sink" to the front.
- Time $O(n^2)$ in all cases (naive version); O(n) best with early-stop flag.
- Stable, in-place, simplest but slowest — rarely used in practice.

3.2 Algorithm

Objective: sort List[1..N] ascending. **Input:** List, N. **Output:** sorted List (in-place).

```
Bubble_Sort(List, N):
1. for I = 1 to N:
```

```
2. for J = 1 to N-I: // last I elements already in place
3. if List[J] > List[J+1]:
4. swap(List[J], List[J+1])
// Optimized: stop if no swap occurs in a full pass (=> already sorted).
```

- **Complexity:** comparisons $C(n) = n + n + ... + n = O(n^2)$; swaps $O(n^2)$ worst.
- **Optimization:** add a `swapped` flag ⇒ best case O(n) for already-sorted input.

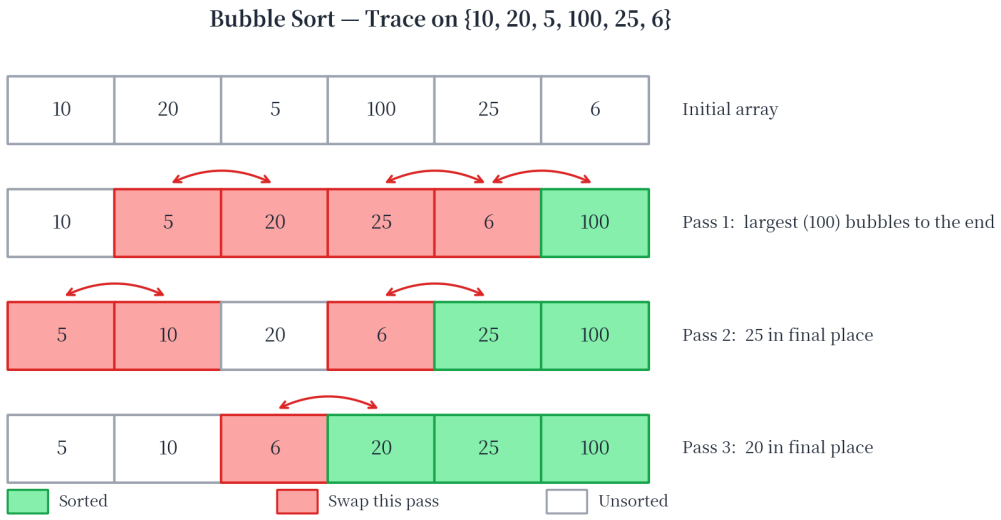


Figure 3.1 — Bubble Sort trace on {10, 20, 5, 100, 25, 6}. After each pass, the next largest element reaches its final position (green). Red highlights swaps in that pass.

3.3 C++ Implementation

```
void bubbleSort(vector& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; ++i) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; ++j) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted
    }
}
```

3.4 Common Mistakes

- Inner loop range: must be `j < n-1-i` (not n-1) — otherwise recompares sorted tail.
- Forgetting early-stop flag wastes a full extra pass on sorted input.
- Using >= instead of > makes it unstable.

4. Selection Sort

4.1 Basic Idea

- Find the smallest element in the unsorted suffix via linear scan; swap it into the next position of the sorted prefix.

- After pass i, A[1..i] holds the i smallest elements in final order.
- Time $O(n^2)$ in all cases (best = average = worst) — comparisons are independent of input order.
- Minimum number of swaps: exactly n-1. Not stable (swap can leapfrog equal keys).

4.2 Algorithm

```
Selection_Sort(List, N):
1. for I = 1 to N:
2. Min = I
3. for J = I+1 to N:
4. if List[J] < List[Min]: Min = J
5. swap(List[I], List[Min])
// Comparisons: (n-1) + (n-2) + ... + 1 = n(n-1)/2 = O(n^2)
```

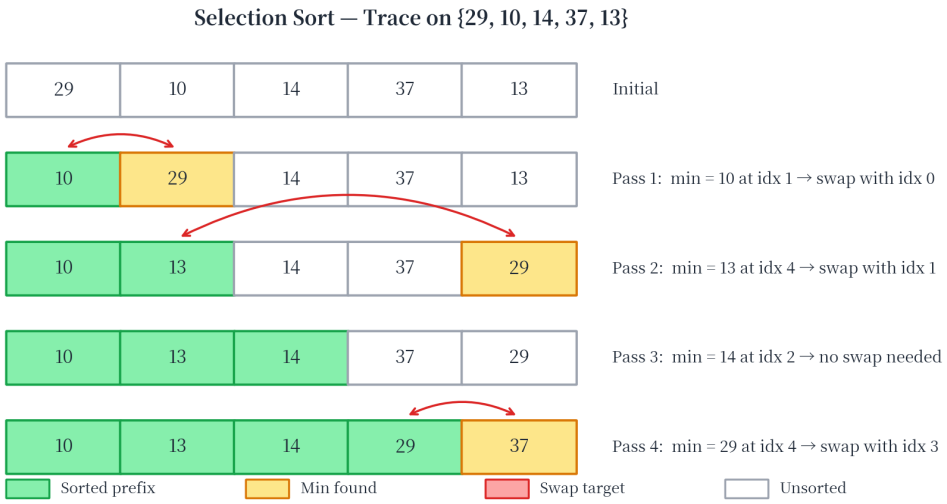


Figure 4.1 — Selection Sort trace on {29, 10, 14, 37, 13}. Each pass scans the unsorted suffix for the minimum (yellow), then swaps it to the boundary (red). Sorted prefix grows by one each pass (green).

4.3 C++ Implementation

```
void selectionSort(vector& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; ++i) {
        int mn = i;
        for (int j = i + 1; j < n; ++j)
            if (a[j] < a[mn]) mn = j;
        swap(a[i], a[mn]);
    }
}
```

4.4 Common Mistakes

- Swapping even when `mn == i` — harmless but wasteful; check first.
- Claiming stability — selection sort is not stable as written (long-distance swap).
- Forgetting that comparisons still happen even when array is sorted — $O(n^2)$ best case.

4.5 Probable Exam Questions (Bubble / Selection / Binary Search)

E Trace Bubble Sort on {10, 20, 5, 100, 25, 6} for 3 passes.

E Why does Selection Sort perform at most n-1 swaps?

E Write iterative Binary Search. Why is `mid = lo + (hi-lo)/2` preferred?

M Prove that Binary Search runs in $\Theta(\lg n)$ time using a recurrence.

M Modify Bubble Sort to terminate early when the array is already sorted. Show best-case complexity.

M Compare the number of swaps in Bubble vs Selection Sort on a reverse-sorted array of size n.

H Given decision-tree lower bound, why can Binary Search achieve O(lg n) but comparison sorts cannot sort faster than Ω(n log n)? Explain the difference.

5. Insertion Sort

5.1 Basic Idea

- Builds a sorted prefix A[1..j-1] one element at a time; inserts A[j] into its correct position by shifting larger elements right.
- Mimics how a human sorts playing cards in their hand.
- Efficient for small or nearly-sorted data — used as a subroutine inside hybrid sorts (TimSort, Introsort).
- Stable and in-place.
- Best case O(n) [already sorted]; Average and Worst $O(n^2)$ [reverse sorted].
- Space O(1); adaptive: number of shifts ≤ number of inversions.

5.2 Algorithm (Cormen pseudocode, A[1..n])

Objective: sort A[1..n] in non-decreasing order. **Input:** array A, size n. **Output:** A sorted in place.

```
INSERTION-SORT(A, n) // A[1..n]
1. for j = 2 to n:
2. key = A[j]
3. i = j - 1
4. while i > 0 and A[i] > key:
5. A[i+1] = A[i] // shift right
6. i = i - 1
7. A[i+1] = key // insert key
```

5.3 Loop Invariant (proof of correctness)

- **Invariant:** At the start of each iteration of the for loop (line 1), the subarray A[1..j-1] consists of the elements originally in A[1..j-1] but in sorted order.
- **Initialization:** j=2 ⟹ A[1..1] has one element ⟹ trivially sorted.
- **Maintenance:** the while loop shifts each A[i] > key one position right, then places key at A[i+1] ⟹ A[1..j] becomes sorted.
- **Termination:** when j = n+1, A[1..n] is sorted = the entire array. (Cormen pp. 18-19.)

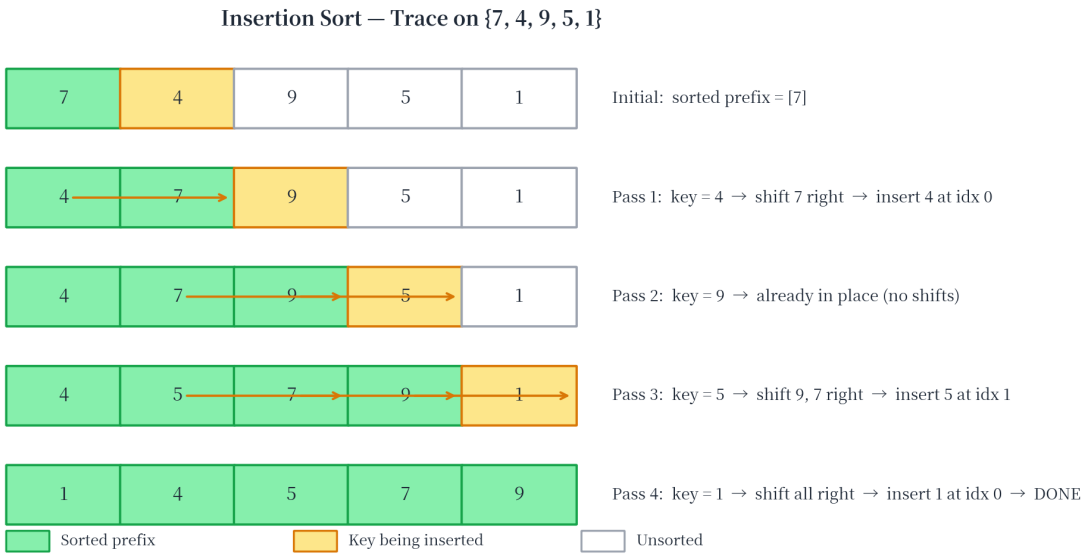


Figure 5.1 — Insertion Sort trace on {7, 4, 9, 5, 1}. The sorted prefix (green) grows by one each pass; the key being inserted (yellow) is compared with each element of the sorted prefix and shifted into place.

5.4 C++ Implementation

```
// 0-indexed version (A[0..n-1])
void insertionSort(vector& a) {
    int n = a.size();
    for (int j = 1; j < n; ++j) {
        int key = a[j];
        int i = j - 1;
        while (i >= 0 && a[i] > key) {
            a[i + 1] = a[i]; // shift right
            --i;
        }
        a[i + 1] = key; // place key
    }
}
```

5.5 Complexity Analysis

- Let t_j = number of while-loop tests for the current j. Then $T(n) = \Sigma\, t_j$, j=2..n.
- **Worst case** (reverse sorted): $t_j = j \Rightarrow T(n) = \Sigma\, j = n(n+1)/2 - 1 = O(n^2)$.
- **Best case** (already sorted): $t_j = 1 \Rightarrow T(n) = \Sigma\, 1 = n-1 = O(n)$.
- **Average case** (random): $t_j \approx j/2 \Rightarrow T(n) = O(n^2)$.
- Space: O(1) auxiliary.

5.6 Important Functions

Function / Step	Purpose	Notes
key = A[j]	Save the element to be inserted	Prevents overwrite during shifting
while A[i] > key	Find insertion point	Strict > preserves stability
A[i+1] = A[i]	Shift larger element right	Creates the gap for key
A[i+1] = key	Insert key into gap	Final placement per outer iteration

5.7 Common Mistakes & Edge Cases

- Overwriting A[j] before saving it as key — always save first.
- Using `>=` in while loop makes the sort unstable.
- Off-by-one in `i >= 0` (0-indexed) vs `i > 0` (1-indexed Cormen style).
- Inserting at i instead of i+1 after the loop — wrong position.
- Forgetting that best case is O(n), not $O(n^2)$ — examiners often ask this.

5.8 Probable Exam Questions

E Dry-run INSERTION-SORT on {7, 4, 9, 5, 1} showing each pass.

E State the loop invariant of Insertion Sort and explain why it holds at termination.

E What is the best-case input for Insertion Sort? What is its complexity?

M Derive T(n) for the worst case (reverse-sorted input) and show it is $O(n^2)$.

M Why is Insertion Sort preferred over Merge Sort for small subarrays ($n \leq 10$)?

M Convert the 1-indexed Cormen pseudocode to a 0-indexed C++ function.

H Show that the running time of Insertion Sort is O(n + I) where I is the number of inversions in the input. Hence argue it is adaptive.

H Implement Insertion Sort on a singly linked list. Identify which steps differ from the array version.

6. Merge Sort

6.1 Basic Idea

- Divide-and-conquer: split the array into two halves, recursively sort each, then merge the two sorted halves.
- All the work is in the merge step (combine); divide is just an index calculation.
- Stable; not in-place — needs Θ(n) auxiliary array.
- Time Θ(n log n) in all cases (best = average = worst).
- Preferred for linked lists and external sorting; predictable performance.

6.2 Algorithm (Cormen pseudocode)

Objective: sort A[p..r] non-decreasing. **Input:** array A, indices p, r. **Output:** A[p..r] sorted.

```
MergeSort(A, p, r)
1. if p < r:
2. q = (p + r) / 2 // divide
3. MergeSort(A, p, q) // conquer left
4. MergeSort(A, q+1, r) // conquer right
5. Merge(A, p, q, r) // combine
// Initial call: MergeSort(A, 0, n-1)

Merge(A, p, q, r) // merge sorted A[p..q] and A[q+1..r]
1. n1 = q - p + 1; n2 = r - q
2. copy A[p..q] into L[1..n1+1]
3. copy A[q+1..r] into R[1..n2+1]
4. L[n1+1] = R[n2+1] = infinity // sentinels
5. i = j = 1
6. for k = p to r:
7. if L[i] <= R[j]: A[k] = L[i]; i++
8. else : A[k] = R[j]; j++
```

- Sentinels (∞) avoid checking whether either subarray is exhausted at each step.
- Merge is Θ(n): n = r-p+1 elements are copied once and merged once.

6.3 C++ Implementation

```
void merge(vector& a, int p, int q, int r, vector& buf) {
    int n1 = q - p + 1, n2 = r - q;
    vector L(a.begin() + p, a.begin() + q + 1);
    vector R(a.begin() + q + 1, a.begin() + r + 1);
    int i = 0, j = 0, k = p;
    while (i < n1 && j < n2)
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++]; // <= keeps stable
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void mergeSort(vector& a, int p, int r) {
    if (p >= r) return;
    int q = p + (r - p) / 2;
    mergeSort(a, p, q);
    mergeSort(a, q + 1, r);
    merge(a, p, q, r);
}
// Driver: mergeSort(a, 0, (int)a.size() - 1);
```

6.4 Recurrence & Complexity

- **Recurrence:** $T(n) = 2 \cdot T(n/2) + \Theta(n)$; $T(1) = \Theta(1)$.
- **Substitution proof:** $T(n) = 2T(n/2) + cn = 2^2T(n/4) + 2cn = \ldots = 2^kT(n/2^k) + k \cdot cn$. Set $2^k = n \Rightarrow k = \lg n$. Then $T(n) = n \cdot T(1) + cn \cdot \lg n = \Theta(n \log n)$.
- **Recursion tree:** depth = lg n; work per level = cn; total = $cn \cdot \lg n + cn = \Theta(n \log n)$.
- Space: $\Theta(n)$ auxiliary (the temp arrays). Stack depth $\Theta(\lg n)$.

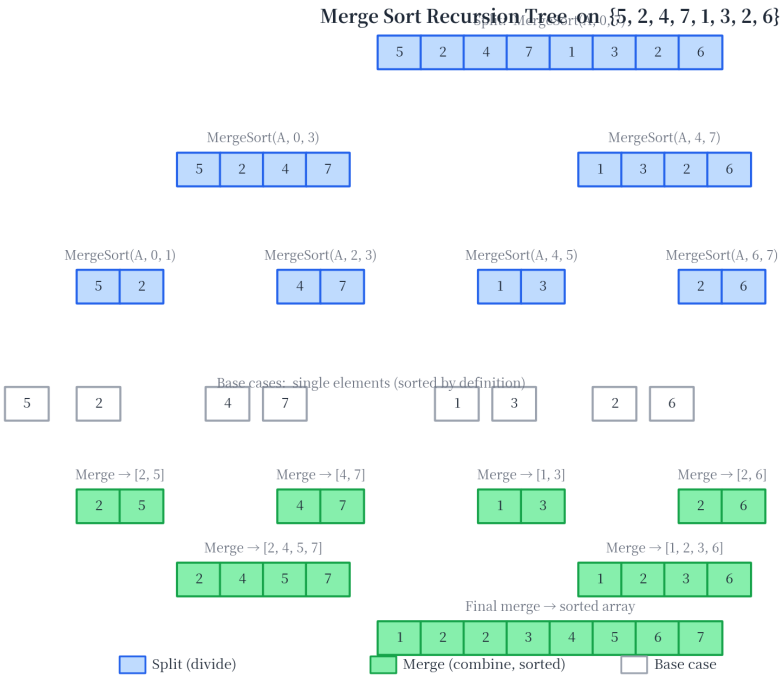


Figure 6.1 — Merge Sort recursion tree on {5, 2, 4, 7, 1, 3, 2, 6}. The array is recursively split (blue) until single elements remain, then merged back (green) in sorted order. Total work per level is Θ(n); there are lg n levels.

6.5 Important Functions

Function / Step	Purpose	Notes

q = (p+r)/2	Compute midpoint	Floor division; p + (r-p)/2 avoids overflow
MergeSort(A,p,q)	Sort left half	Recursion depth Θ(lg n)
MergeSort(A,q+1,r)	Sort right half	Symmetric to left
Merge(A,p,q,r)	Combine two sorted halves	Θ(n) time, Θ(n) space
Sentinel ∞	Avoid empty-check per step	Simplifies merge loop

6.6 Bonus Variant: Counting Inversions

- **Problem:** count pairs (i, j) with i < j but A[i] > A[j] (out-of-order pairs). Used in recommendation systems to compare two ranked lists.
- **Idea:** extend Merge — when taking an element from the right subarray R[j], it is inverted with every remaining element in L (count += n1 - i + 1).
- **Complexity:** same as Merge Sort: Θ(n log n).

```
// Returns inversion count in A[p..r]; sorts A as a side effect.
long long countInv(vector& a, int p, int r) {
    if (p >= r) return 0;
    int q = p + (r - p) / 2;
    long long cnt = countInv(a, p, q) + countInv(a, q + 1, r);
    // merge-and-count:
    vector L(a.begin() + p, a.begin() + q + 1);
    vector R(a.begin() + q + 1, a.begin() + r + 1);
    int i = 0, j = 0, k = p, n1 = L.size();
    while (i < (int)L.size() && j < (int)R.size()) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else { a[k++] = R[j++]; cnt += n1 - i; } // inversions!
    }
    while (i < (int)L.size()) a[k++] = L[i++];
    while (j < (int)R.size()) a[k++] = R[j++];
    return cnt;
}
```

6.7 Common Mistakes

- Forgetting to copy back from the temp buffer to A — result vanishes.
- Using < instead of ≤ in merge — breaks stability.
- Wrong midpoint formula causes infinite recursion when r = p+1.
- Allocating temp arrays inside merge (O(n log n) total allocations) — better to pass a single reusable buffer.

6.8 Probable Exam Questions

E Sort {5, 2, 4, 7, 1, 3, 2, 6} using Merge Sort; draw the recursion tree.

E State the recurrence for Merge Sort and its solution.

E Why is Merge Sort stable but Quick Sort not?

M Prove T(n) = 2T(n/2) + cn = Θ(n log n) using the substitution method.

M Why does Merge Sort need Θ(n) extra space? Can it be made in-place?

M Modify Merge to count inversions. State the recurrence and final complexity.

H Why is Merge Sort preferred over Quick Sort for sorting linked lists? Give an in-place merge for linked lists.

H Compare Merge Sort and Quick Sort on: (i) worst-case time, (ii) space, (iii) stability, (iv) cache behaviour. Which would you choose for external sorting on disk, and why?

7. Quick Sort

7.1 Basic Idea

- In-place divide-and-conquer: pick a pivot, partition the array so that elements ≤ pivot are on its left and elements > pivot are on its right, then recurse on each side.
- Combine step is empty — sorting happens entirely via partitioning (unlike Merge Sort).
- Best / Average case: $\Theta(n \log n)$. Worst case: $\Theta(n^2)$ [already-sorted input with first/last pivot].
- Not stable; in-place (O(lg n) stack space).
- In practice, the fastest general-purpose sort due to small constants and good cache behaviour.

7.2 Algorithm (Lomuto Partition, Cormen)

Objective: sort A[p..r] in place. **Input:** array A, indices p, r. **Output:** A[p..r] sorted.

```
QUICKSORT(A, p, r)
1. if p < r:
2. q = PARTITION(A, p, r)
3. QUICKSORT(A, p, q-1)
4. QUICKSORT(A, q+1, r)
// Initial call: QUICKSORT(A, 0, n-1)

PARTITION(A, p, r) // pivot = A[r]
1. x = A[r] // pivot value
2. i = p - 1 // end of '<= x' region
3. for j = p to r-1:
4. if A[j] <= x:
5. i = i + 1
6. swap A[i] with A[j]
7. swap A[i+1] with A[r] // place pivot in final position
8. return i + 1 // pivot index
```

- **Partition invariant:** at any point during the loop, A[p..i] ≤ pivot, A[i+1..j-1] > pivot, A[j..r-1] unprocessed, A[r] = pivot.
- **Partition cost:** $\Theta(n)$ (one linear scan).

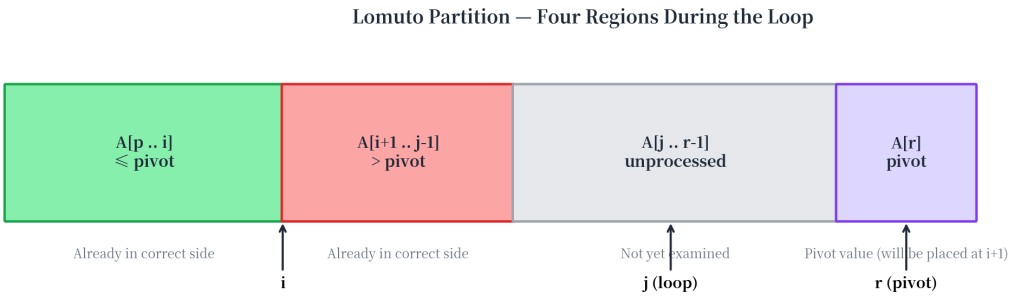


Figure 7.1 — Lomuto partition invariant. Four regions are maintained during the scan: elements ≤ pivot (green), elements > pivot (red), unprocessed elements (grey), and the pivot at the end (purple).

7.3 Alternative: Hoare-style Partition (pivot = A[left])

- Used in the slides (34-quicksort). Two pointers l, r approach from each end and swap out-of-place pairs.

```
Partition(A, left, right): // pivot = A[left]
1. pivot = A[left]; l = left+1; r = right
2. while l < r:
3. while l < right and A[l] < pivot: l++
4. while r > left and A[r] >= pivot: r--
```

```
5. if l < r: swap A[l] with A[r]
6. A[left] = A[r]; A[r] = pivot
7. return r
```

7.4 C++ Implementation (Lomuto, exam-friendly)

```
int partition(vector& a, int p, int r) {
    int x = a[r]; // pivot = last element
    int i = p - 1;
    for (int j = p; j < r; ++j) {
        if (a[j] <= x) {
            ++i;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[r]);
    return i + 1; // final pivot index
}

void quickSort(vector& a, int p, int r) {
    if (p < r) {
        int q = partition(a, p, r);
        quickSort(a, p, q - 1);
        quickSort(a, q + 1, r);
    }
}
// Driver: quickSort(a, 0, (int)a.size() - 1);
```

7.5 Randomized Quick Sort

- **Motivation:** deterministic pivot (first / last) gives $O(n^2)$ on sorted input. Randomizing the pivot makes the worst case extremely unlikely.
- **Method:** pick a random index $i \in [p, r]$, swap A[i] with A[r], then run ordinary PARTITION.

```
int randomizedPartition(vector& a, int p, int r) {
    int i = p + rand() % (r - p + 1); // random index in [p, r]
    swap(a[i], a[r]); // move random pivot to end
    return partition(a, p, r); // then standard Lomuto
}

void randomizedQuickSort(vector& a, int p, int r) {
    if (p < r) {
        int q = randomizedPartition(a, p, r);
        randomizedQuickSort(a, p, q - 1);
        randomizedQuickSort(a, q + 1, r);
    }
}
```

7.6 Best / Worst / Average Case

- **Best case:** pivot splits the array in half every time $\Rightarrow T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- **Worst case:** pivot is the smallest or largest element every time (e.g., sorted input + first/last pivot) $\Rightarrow T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.
- **Average case (random pivot):** expected $\Theta(n \log n)$ — provable via indicator random variables (Cormen §7.4).

7.7 Quick Sort Recursion Tree

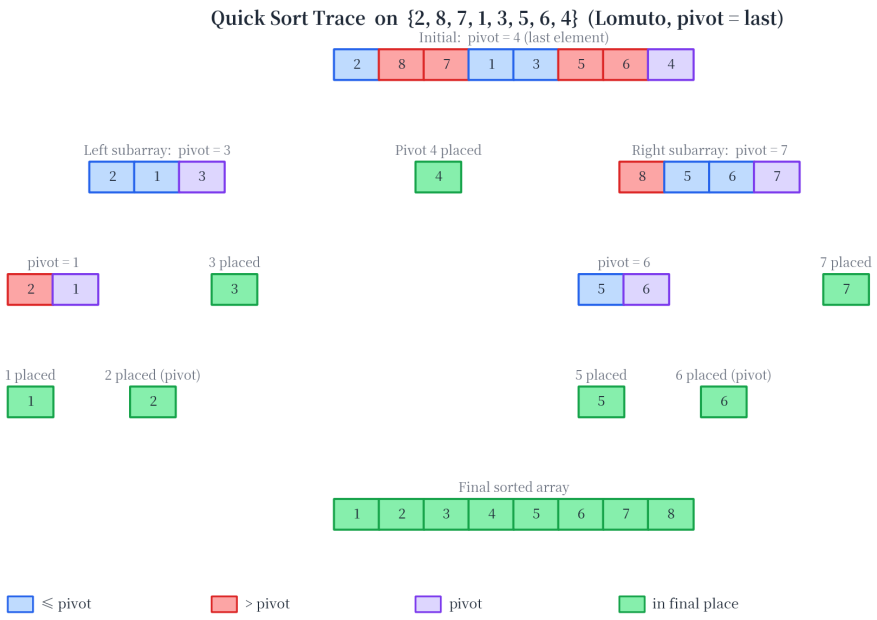


Figure 7.2 — Quick Sort trace on {2, 8, 7, 1, 3, 5, 6, 4} using Lomuto partition with pivot = last. Each level shows the subarray, the pivot (purple), and the partition into ≤ pivot (blue) and > pivot (red) sides. Pivot reaches its final position at every level (green).

7.8 Merge Sort vs Quick Sort

Aspect	Merge Sort	Quick Sort	Winner
Worst-case time	$\Theta(n \log n)$	$\Theta(n^2)$	Merge
Average time	$\Theta(n \log n)$	$\Theta(n \log n)$	Tie (Quick has lower constants)
Auxiliary space	$\Theta(n)$	$O(\lg n)$ stack	Quick
Stable	Yes	No	Merge
In-place	No	Yes	Quick
Cache friendly	No (random access)	Yes (sequential)	Quick
Works on linked lists	Yes (no random access needed)	No (needs random access)	Merge
Combine step cost	$\Theta(n)$ merge	Empty (work in divide)	—

7.9 Common Mistakes

- Pivot = first/last element on already-sorted input ⇒ guaranteed $\Theta(n^2)$.
- Off-by-one in partition indices (i+1 vs i) — misplaces pivot.
- Forgetting base case `if p < r` ⇒ infinite recursion.
- Recursing on (p, q) and (q, r) instead of (p, q-1) and (q+1, r) ⇒ pivot re-enters, infinite loop.
- Using < instead of ≤ in partition — may cause uneven splits when many duplicates exist.

7.10 Probable Exam Questions

E Trace PARTITION on {2, 5, 8, 3, 9, 4, 1, 7, 10, 6} with pivot = 6 (last element).

E State the recurrence for Quick Sort in best and worst case.

E When does Quick Sort degrade to $O(n^2)$? How is this avoided?

M Implement Randomized-Partition. Why does randomization not change worst-case bound but improves expected bound?

M Trace Median-of-Three pivot selection on a sorted array of 10 elements. Show the resulting partition.

M Compare Merge Sort and Quick Sort on (a) stability, (b) space, (c) worst-case time, (d) practical performance.

H Prove that the expected running time of Randomized Quick Sort is O(n log n). (Hint: use indicator random variables for each pair of elements being compared.)

H Modify Quick Sort so that its worst-case stack space is O(lg n) even on sorted input. (Hint: tail-recursion elimination — recurse on the smaller side first.)

H Given an array of n identical elements, what is the running time of the Lomuto-partition Quick Sort as written above? Modify the partition to handle duplicates efficiently (3-way partition / Dutch National Flag).

8. Counting Sort

8.1 Basic Idea

- Non-comparison sort that exploits knowledge of the value range k.
- Counts how many input elements have each value, then computes prefix sums to determine each element's final position.
- Time $\Theta(n + k)$ where k = max value — linear when k = O(n).
- Stable (process input from right to left during placement).
- Impractical when $k \gg n$ (memory + time blow up).

8.2 Algorithm

Input: A[1..n] with values in [0, k]. **Output:** B[1..n] sorted. **Aux:** C[0..k].

```
Counting-Sort(A, B, k):
1. for i = 0 to k: C[i] = 0 // init counter
2. for j = 1 to n: C[A[j]]++ // count occurrences
3. for i = 1 to k: C[i] += C[i-1] // prefix sums: C[i] = #elements <= i
4. for j = n downto 1: // reverse => stable
5. B[C[A[j]]] = A[j]
6. C[A[j]]-- // decrement so next equal element goes one slot left
```

- **Step 1:** init C — O(k).
- **Step 2:** count occurrences — O(n).
- **Step 3:** prefix sum — O(k). Now C[i] = number of elements ≤ i.
- **Step 4-6:** place from right to left — O(n). Reverse traversal is what makes it stable.

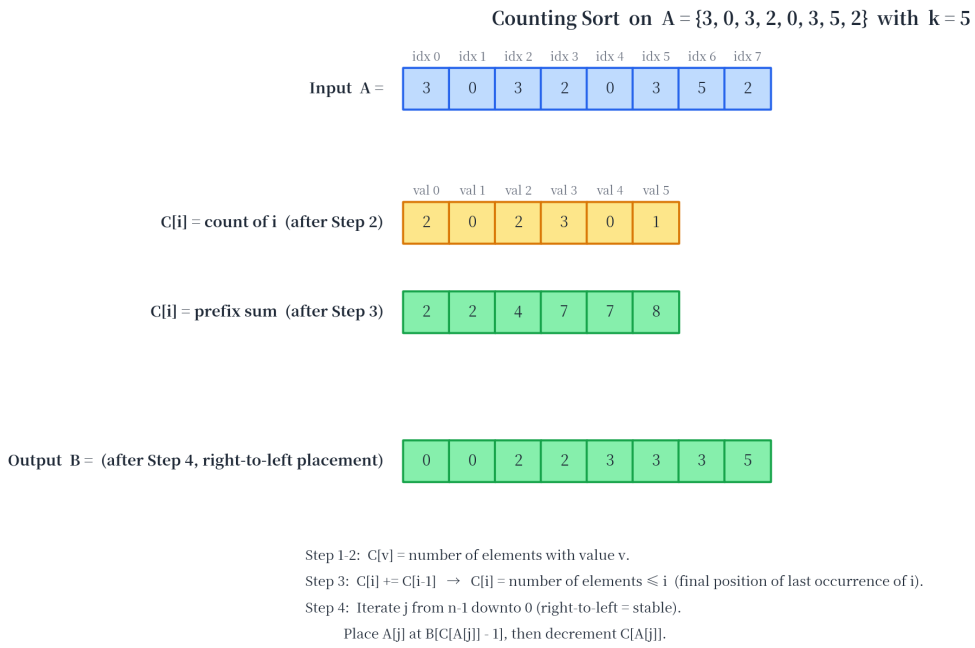


Figure 8.1 — Counting Sort on A = {3, 0, 3, 2, 0, 3, 5, 2} with k = 5. The count array C is built in two stages (raw counts, then prefix sum). Output B is filled right-to-left to preserve stability.

8.3 C++ Implementation

```
// Sorts a (values in [0, k]) into b. Requires k = max(a).
void countingSort(const vector& a, vector& b, int k) {
    int n = a.size();
    vector C(k + 1, 0);
    for (int x : a) C[x]++; // count
    for (int i = 1; i <= k; ++i) C[i] += C[i - 1]; // prefix sum
    b.resize(n);
    for (int j = n - 1; j >= 0; --j) { // reverse => stable
        b[--C[a[j]]] = a[j];
    }
}

// Convenience wrapper
vector countingSort(vector a) {
    if (a.empty()) return a;
    int k = *max_element(a.begin(), a.end());
    vector b;
    countingSort(a, b, k);
    return b;
}
```

8.4 Important Functions & Complexity

Step	Purpose	Time
C[i] = 0 (init)	Reset counter array	O(k)
C[A[j]]++	Count occurrences of each value	O(n)
C[i] += C[i-1]	Prefix sum: positions in B	O(k)
B[C[A[j]]] = A[j]	Place each element in final spot	O(n)

Total	—	Θ(n + k)

- Space: O(n + k) — needs B and C.
- Linear iff k = O(n). If k = $O(n^2)$, total = $O(n^2)$ — worse than Merge Sort.

8.5 Common Mistakes

- Iterating placement step left-to-right — breaks stability. Must go right-to-left.
- Forgetting to decrement C[A[j]] after placement — all equal keys land in the same slot.
- Not handling negative keys — must offset by min value.
- Using C of size n instead of k+1 — out-of-bounds when max value > n.

9. Radix Sort

9.1 Basic Idea

- Sorts integers (or strings) by processing one digit (or character) at a time, using a stable sort (usually Counting Sort) at each digit.
- Two variants: **LSD** (Least Significant Digit first) — process from rightmost digit leftward; **MSD** (Most Significant Digit first) — process from leftmost digit rightward. LSD is more common.
- Number of passes = number of digits d in the maximum key. Each pass uses Counting Sort on base b.
- Time O(d · (n + b)) where d = #digits, b = base. Space O(n + b).
- Stable (mandatory: stability of the inner sort is what makes Radix Sort correct).

9.2 Algorithm (LSD Radix Sort)

Input: A[1..n] integers, each with at most d digits in base b. **Output:** A sorted ascending.

```
Radix-Sort(A, d, b): // d = #digits, b = base (e.g., 10)
1. for i = 1 to d: // from LSD to MSD
2. stable-sort A by digit i // usually Counting Sort on digit i only
```

- **Why stable inner sort?** so that the order produced by earlier (less significant) digit passes is preserved for equal keys in the current pass.
- **Example** (slides): {348, 143, 361, 423, 53, 128, 321, 543, 366, 202} ⟹ pass on units ⟹ pass on tens ⟹ pass on hundreds ⟹ {53, 128, 143, 202, 321, 348, 361, 366, 423, 543}.

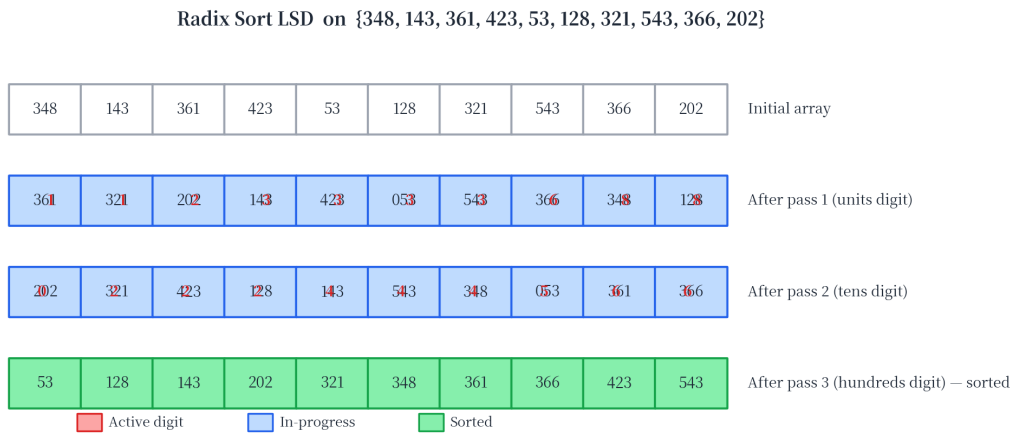


Figure 9.1 — LSD Radix Sort on {348, 143, 361, 423, 53, 128, 321, 543, 366, 202}. Numbers are padded with leading zeros for display. The active digit at each pass is highlighted in red; the array is fully sorted after the hundreds pass (green).

9.3 C++ Implementation (decimal LSD)

```
// LSD Radix Sort for non-negative integers, base 10.
void radixSort(vector& a) {
    if (a.empty()) return;
    int mx = *max_element(a.begin(), a.end());
    for (int exp = 1; mx / exp > 0; exp *= 10) {
        countingByDigit(a, exp); // stable sort on current digit
    }
}

// Stable counting sort on the digit at position 'exp' (1, 10, 100, ...).
void countingByDigit(vector& a, int exp) {
    int n = a.size();
    vector out(n);
    vector cnt(10, 0);
    for (int x : a) cnt[(x / exp) % 10]++;
    for (int i = 1; i < 10; ++i) cnt[i] += cnt[i - 1];
    for (int j = n - 1; j >= 0; --j) { // reverse => stable
        int d = (a[j] / exp) % 10;
        out[--cnt[d]] = a[j];
    }
    a.swap(out);
}
```

9.4 Complexity & Common Mistakes

Case	#digits s	Total time C(n)
Best	$s = \log_b n$	O(n log n)
Worst	s = n	$O(n^2)$
Typical (fixed d)	s = d (constant)	$O(d \cdot (n + b)) \approx O(n)$

- Inner sort MUST be stable — using an unstable sort breaks Radix Sort completely.
- Forgetting to handle leading zeros (pad shorter numbers).
- Choosing base b too large ⟹ cnt array and passes shrink but memory grows; b too small ⟹ more passes.
- Not handling negative numbers — split into negatives and positives, sort each, then concatenate.

10. Decision-Tree Lower Bound (recap with proof)

- For an n-element array, a comparison sort corresponds to a binary decision tree where each internal node is a comparison ($a \le b$?) and each leaf is a permutation that the algorithm outputs.
- The tree must have at least n! leaves (one per valid permutation).
- Worst-case comparisons = tree height h. Max leaves in a binary tree of height h is 2^h.
- Hence $n! \le 2^h \Rightarrow h \ge \lg(n!)$.
- Using Stirling-like bound: $\lg(n!) = \Omega(n \log n)$ — specifically, $\lg(n!) \ge (n/2) \cdot \lg(n/2) = \Omega(n \log n)$.
- Conclusion: any comparison sort needs $\Omega(n \log n)$ comparisons in the worst case. Merge Sort, Heap Sort are asymptotically optimal.
- Why Counting/Radix escape: they are not comparison sorts — they use the actual key value as an array index, so the decision-tree argument does not apply.

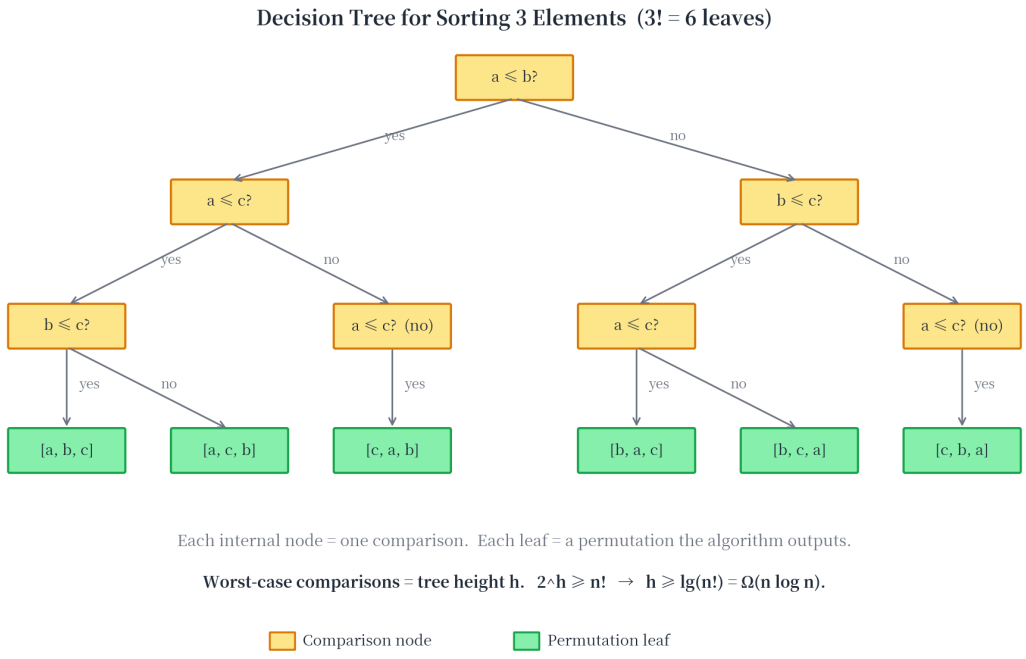


Figure 10.1 — Decision tree for sorting 3 elements has 3! = 6 leaves (one per permutation). The tree height $h \geq \lg(6) \approx 2.58$, so at least 3 comparisons are needed in the worst case. In general, $h \geq \lg(n!) = \Omega(n \log n)$.

10.1 Comparison Sorts vs Non-Comparison Sorts

Aspect	Comparison Sorts	Non-Comparison Sorts	Notes
Lower bound	Ω(n log n)	None (can be O(n))	Counting, Radix, Bucket
Input assumption	Any comparable keys	Integer keys (or known range)	Restricted domain
Examples	Merge, Quick, Heap, Bubble, Insertion, Selection	Counting, Radix, Bucket	—
Stability	Varies	Usually stable	Radix requires stable inner sort
Adaptivity	Some (Insertion)	No	—

10.2 Probable Exam Questions (Counting / Radix / Lower Bound)

E Trace Counting Sort on A = {3, 0, 3, 2, 0, 3, 5, 2} with k = 5. Show C after each step.

E Why must the placement loop in Counting Sort run from n downto 1?

E Sort {348, 143, 361, 423, 53, 128, 321, 543, 366, 202} using LSD Radix Sort; show the array after each pass.

M Prove that any comparison sort requires Ω(n log n) comparisons in the worst case.

M What is the time complexity of Counting Sort when $k = O(n^2)$? When is Counting Sort preferred over Merge Sort?

M Why must the inner sort in Radix Sort be stable? Show with a 2-digit example what breaks if it is not.

H Design a Radix Sort for an array of n integers in the range $[0, n^3 - 1]$ that runs in O(n) time. Choose the base carefully and justify.

H How would you sort n integers in the range [1, n^2] in O(n) time? Can this be done with a comparison sort? Why or why not?

11. Formula & Complexity Sheet (Quick Revision)

11.1 Complete Complexity Reference

Algorithm	Best	Average	Worst	Space	Stable?	Recurrence / Key
Binary Search	Θ(1)	Θ(lg n)	Θ(lg n)	O(1)	n/a	T(n) = T(n/2) + Θ(1)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	(n-1) + (n-2) + ... + 1
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	n(n-1)/2 comparisons
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	T(n) = Σ t_j; reverse ⟹ n(n-1)/2
Merge Sort	O(n lg n)	O(n lg n)	O(n lg n)	O(n)	Yes	T(n) = 2T(n/2) + Θ(n)
Quick Sort	O(n lg n)	O(n lg n)	$O(n^2)$	O(lg n)	No	Best: 2T(n/2)+Θ(n); Worst: T(n-1)+Θ(n)
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(n+k)	Yes	Non-comparison; linear iff k = O(n)
Radix Sort	O(d(n+b))	O(d(n+b))	O(d(n+b))	O(n+b)	Yes	LSD: d stable passes of Counting Sort

11.2 Key Formulas to Memorize

- Sum 1..n = n(n+1)/2 ; Sum 1..(n-1) = n(n-1)/2 — basis for $O(n^2)$ sorts.
- Stirling: lg(n!) = n·lg n - O(n) = Θ(n log n) — basis of comparison-sort lower bound.
- Master theorem (D&C;): T(n) = a·T(n/b) + f(n). For Merge Sort: a=2, b=2, f(n)=Θ(n) ⟹ Θ(n log n).
- Binary Search recurrence: T(n) = T(n/2) + Θ(1) = Θ(lg n).
- Quick Sort best: T(n) = 2T(n/2) + Θ(n) = Θ(n log n).
- Quick Sort worst: T(n) = T(n-1) + Θ(n) = Θ(n^2).
- Counting Sort: T(n) = Θ(n + k); linear iff k = O(n).
- Radix Sort: T(n) = O(d·(n + b)); worst case s = n ⟹ $O(n^2)$; best case s = $\log_b$ n ⟹ O(n log n).

11.3 Comparison Tables

(a) Bubble vs Selection vs Insertion

Aspect	Bubble	Selection	Insertion
Best time	O(n) [with flag]	$O(n^2)$	O(n)
Worst time	$O(n^2)$	$O(n^2)$	$O(n^2)$
Swaps (worst)	$O(n^2)$	n-1 (minimum)	$O(n^2)$ shifts

Stable	Yes	No	Yes
In-place	Yes	Yes	Yes
Adaptive	Yes (with flag)	No	Yes (shifts = inversions)
Best for	Nearly sorted, teaching	Minimizing swaps	Small / nearly sorted

(b) Recursive vs Iterative Implementation

Aspect	Recursive	Iterative
Code length	Shorter, mirrors pseudocode	Longer, explicit stack
Space	Call stack O(depth)	Explicit stack O(depth)
Stack overflow risk	Yes for deep recursion (e.g. Quick Sort worst)	No
Used for	Merge Sort, Quick Sort, Binary Search	Binary Search, Bubble, Selection, Insertion
Tail-recursion optimizable	Sometimes (Quick Sort)	N/A

11.4 Consolidated Probable Exam Questions

Theory

E Define a comparison sort. Give two examples of non-comparison sorts.

E Define stability of a sorting algorithm. Why does it matter?

E Define in-place sorting. Which of Merge / Quick / Insertion are in-place?

M Explain the Divide & Conquer paradigm. Which sorting algorithms use it?

M Why is the lower bound for comparison sorts Ω(n log n)? Give the decision-tree argument.

M Why is Quick Sort usually faster than Merge Sort in practice despite same asymptotic average?

H Compare Best / Average / Worst case of all 6 comparison sorts. Which would you use for: (i) nearly-sorted data, (ii) data with many duplicates, (iii) linked lists, (iv) external sorting?

Algorithms & Tracing

E Write the algorithm for Insertion Sort. State its loop invariant.

E Write the algorithm for Partition (Lomuto). Identify the four regions.

M Trace Merge Sort on {67, 45, 23, 14, 6, 33, 98, 42}. Show the recursion tree and merge steps.

M Trace Quick Sort (pivot = last) on {2, 8, 7, 1, 3, 5, 6, 4}. Show each partition.

M Trace Counting Sort on {4, 1, 3, 4, 3, 1, 2} with k = 4.

H Trace Randomized Quick Sort on a sorted array of 6 elements, assuming the random pivot chosen is always the middle element. Analyze the resulting partition sizes.

Coding & Bug-Finding

E Implement Bubble Sort with an early-termination optimization.

E Implement Selection Sort on a vector.

M Implement Merge Sort iteratively (bottom-up) without recursion.

M Implement Randomized Quick Sort in C++. Why is it preferred over deterministic Quick Sort?

M Implement Counting Sort for an array that may contain negative integers.

M Implement Radix Sort (LSD, base 10) for non-negative integers.

H The following Quick Sort has a bug. Identify and fix it: `if (p <= r) { q = partition(a,p,r); qs(a,p,q); qs(a,q+1,r); }`

H Predict the output of Counting Sort if step 4 iterates from 1 to n (instead of n downto 1). Is the result still sorted? Is it still stable?

H Modify Merge Sort to count inversions. Submit a complete C++ implementation and analyze its complexity.

12. One-Page Final Revision (5-minute pre-exam)

Read this top to bottom in under 5 minutes. Memorize every line.

Key Definitions (one-liners)

- **Comparison sort:** sorts using only pairwise comparisons; Ω(n log n) lower bound.
- **Stable sort:** equal keys retain their original relative order.
- **In-place sort:** uses O(1) (or O(lg n)) auxiliary space.
- **Adaptive sort:** runs faster on nearly-sorted input (e.g. Insertion).
- **Divide & Conquer:** Divide ⇒ Conquer (recurse) ⇒ Combine. Used by Binary Search, Merge Sort, Quick Sort.
- **Pivot:** element chosen to partition Quick Sort. Bad pivot ⇒ $O(n^2)$.
- **Loop Invariant:** property true at start of each loop iteration; used to prove correctness (Insertion Sort: A[1..j-1] sorted).

Complexities (memorize this table)

Algorithm	Best	Avg	Worst	Space	Stable	In-place
Binary Srch	Θ(1)	Θ(lg n)	Θ(lg n)	O(1)	—	Yes
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Yes
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes
Merge	O(n lg n)	O(n lg n)	O(n lg n)	O(n)	Yes	No
Quick	O(n lg n)	O(n lg n)	$O(n^2)$	O(lg n)	No	Yes
Counting	O(n+k)	O(n+k)	O(n+k)	O(n+k)	Yes	No
Radix	O(d(n+b))	O(d(n+b))	O(d(n+b))	O(n+b)	Yes	No

Critical Formulas & Recurrences

- Master (Merge / Quick best): T(n) = 2T(n/2) + Θ(n) = Θ(n log n).
- Quick worst: T(n) = T(n-1) + Θ(n) = $\Theta(n^2)$.
- Binary Search: T(n) = T(n/2) + Θ(1) = Θ(lg n).
- Sum 1..n = n(n+1)/2 (basis of Insertion / Bubble / Selection worst case).
- Lower bound: lg(n!) = Ω(n log n) — any comparison sort.
- Counting linear iff k = O(n). Otherwise Θ(n+k) can be $\Theta(n^2)$.
- Radix: O(d·(n+b)); d = digits, b = base; needs stable inner sort.

Quick Sort Pivot Rules

- Pivot = first or last ⇒ $O(n^2)$ on sorted / reverse-sorted input.

- Pivot = random ⟹ expected O(n log n); worst case still $O(n^2)$ but extremely unlikely.
- Pivot = median-of-three (first, middle, last) ⟹ robust in practice; can pre-sort the 3 to simplify partition loop.
- Lomuto partition: pivot = A[r], i = p-1, scan j from p to r-1, swap A[i+1] with A[r] at end.
- Hoare partition: pivot = A[left], two pointers l (left+1) and r (right) converging.

Merge Sort Key Rules

- Midpoint: q = p + (r-p)/2 (overflow-safe).
- Merge uses sentinels (∞) to avoid empty-subarray checks.
- Merge is Θ(n) — single linear pass; stability requires L[i] ≤ R[j].
- Counting Inversions: when taking R[j], add (n1 - i) to count — same recurrence as Merge Sort.

Common Pitfalls (exam favourites)

- Integer overflow in (lo+hi)/2 ⟹ use lo+(hi-lo)/2.
- Infinite recursion in Quick Sort: missing if p < r, or recursing on (p, q) instead of (p, q-1).
- Counting Sort placement left-to-right ⟹ unstable; must be right-to-left.
- Forgetting to decrement C[A[j]] after placement ⟹ all equal keys collide.
- Radix Sort with non-stable inner sort ⟹ wrong answer.
- Merge Sort forgetting to copy back from temp buffer ⟹ result lost.
- Insertion Sort using >= instead of > ⟹ unstable.
- Bubble Sort inner loop range: must be n-1-i, not n-1.
- Selection Sort: claiming it is stable — it is not as written.

Memorize Before Entering the Hall

- The complexity table above — best/avg/worst for all 8 algorithms.
- The recurrence T(n) = 2T(n/2) + Θ(n) = Θ(n log n) and how to solve it via substitution.
- The decision-tree lower bound proof: n! leaves ⟹ height ≥ lg(n!) = Ω(n log n).
- Which sorts are stable: Bubble, Insertion, Merge, Counting, Radix — not Selection, not Quick.
- Which sorts are in-place: Bubble, Selection, Insertion, Quick — not Merge, not Counting, not Radix.
- Quick Sort worst case trigger: sorted/reverse-sorted input with first/last pivot.
- Counting Sort linear iff k = O(n); Radix Sort needs stable inner sort.
- Insertion Sort loop invariant: at start of each for iteration, A[1..j-1] is sorted.
- Binary Search uses mid = lo + (hi-lo)/2 to avoid overflow.
- Merge Sort auxiliary space Θ(n); Quick Sort auxiliary space O(lg n) stack.

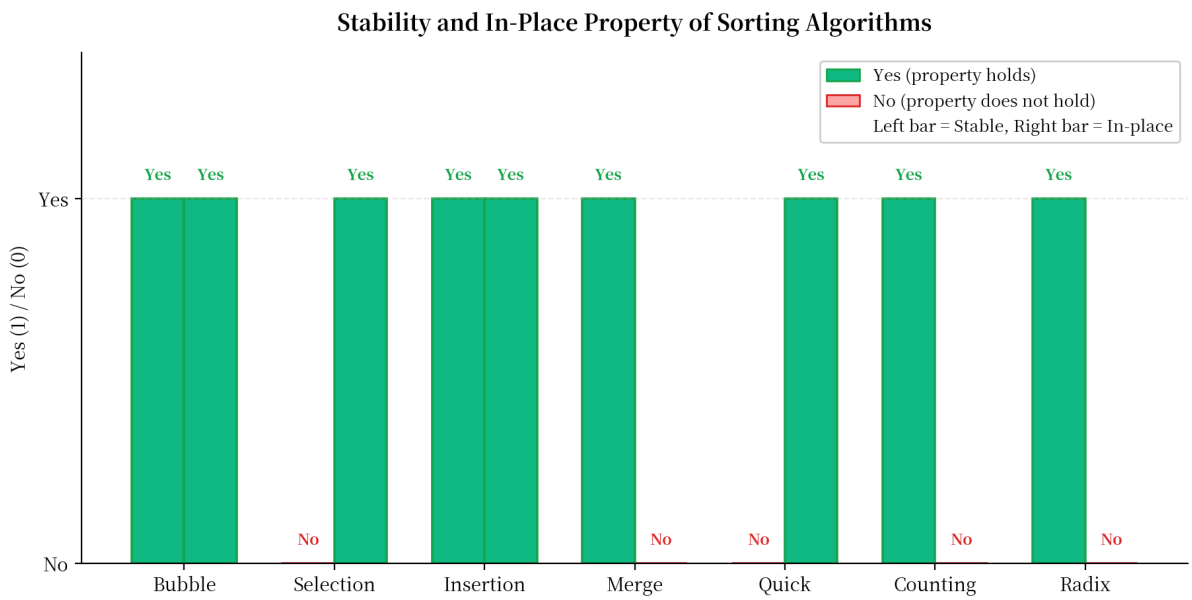


Figure 12.1 — Quick visual summary of stability and in-place properties. Note: Counting and Radix Sorts are stable but NOT in-place (need extra array); Quick is in-place but NOT stable.